

Treap

Treap

前置知识: [朴素二叉搜索树](#), [堆基础](#)。

简介

Treap (树堆) 是一种 **弱平衡** 的二叉搜索树。

Treap 的结点除了被维护的 **权值** () 之外, 还附加了一个随机的 **优先级** ()。其中, 权值满足二叉搜索树性质, 优先级满足堆性质 (小根堆或大根堆)。

其中, 二叉搜索树的性质是指:

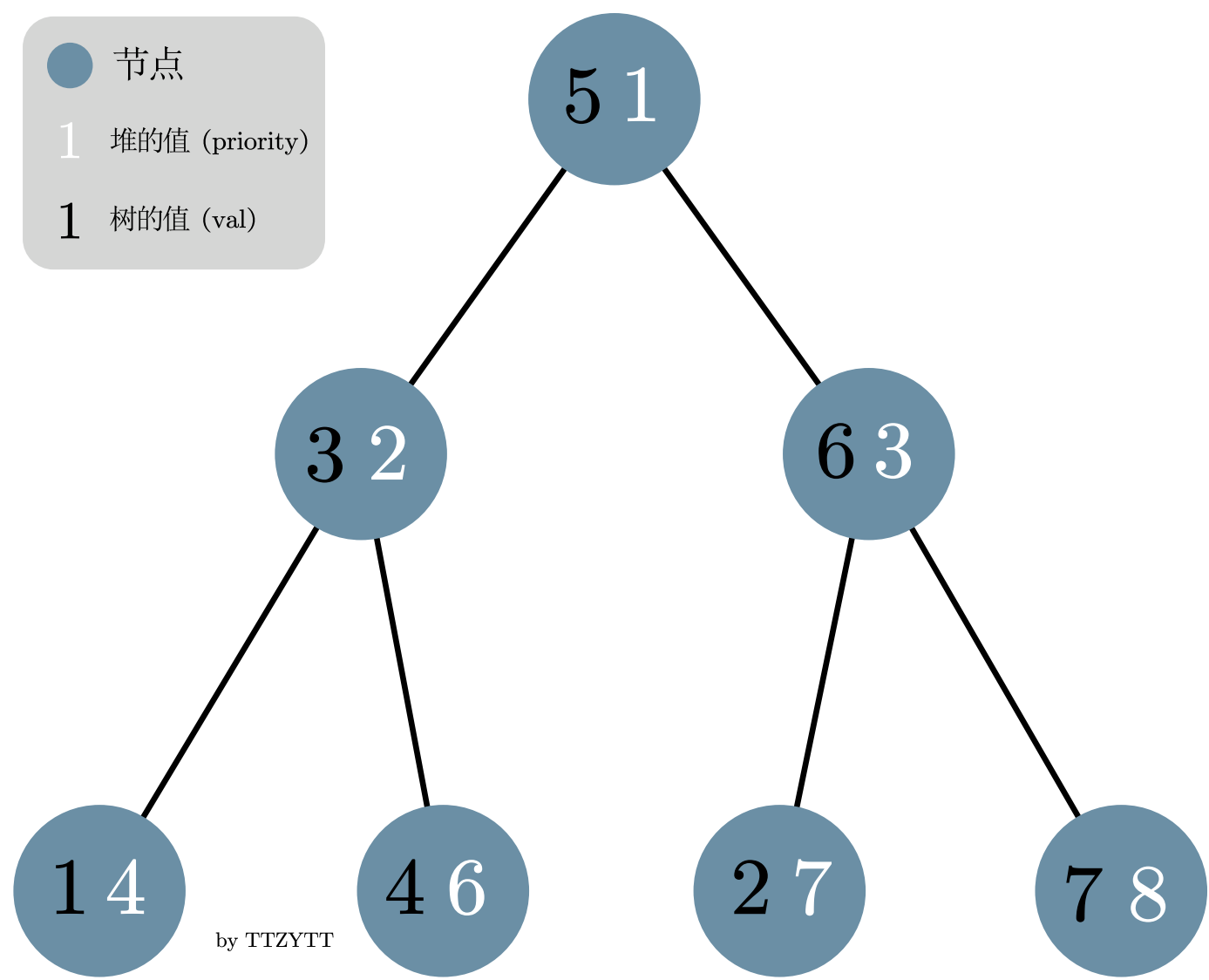
- 左子节点的权值 () 比父节点小。
- 右子节点的权值 () 比父节点大。

堆的性质是:

- 子节点优先级 () 比父节点大或小 (取决于是小根堆还是大根堆)。

不难看出, 如果用的是同一个值, 那么这两种数据结构在组合后会变成一条链, 所以我们再在搜索树的基础上, 引入一个给堆的值。对于 值, 我们维护搜索树的性质, 对于 值, 我们维护堆的性质。其中 这个值是随机给出的。

下图就是一个 Treap 的例子 (这里使用的是小根堆, 即根节点的优先级最小)。



那我们为什么需要大费周章的去让这个数据结构符合树和堆的性质, 并且随机给出堆的值呢?

要理解这个, 首先需要理解朴素二叉搜索树的问题。在给朴素搜索树插入一个新节点时, 我们需要从这个搜索树的根节点开始递归, 如果新节点比当前节点小, 那就向左递归, 反之亦然。

最后当发现当前节点没有子节点时, 就根据新节点的值的大小, 让新节点成为当前节点的左或右子节点。

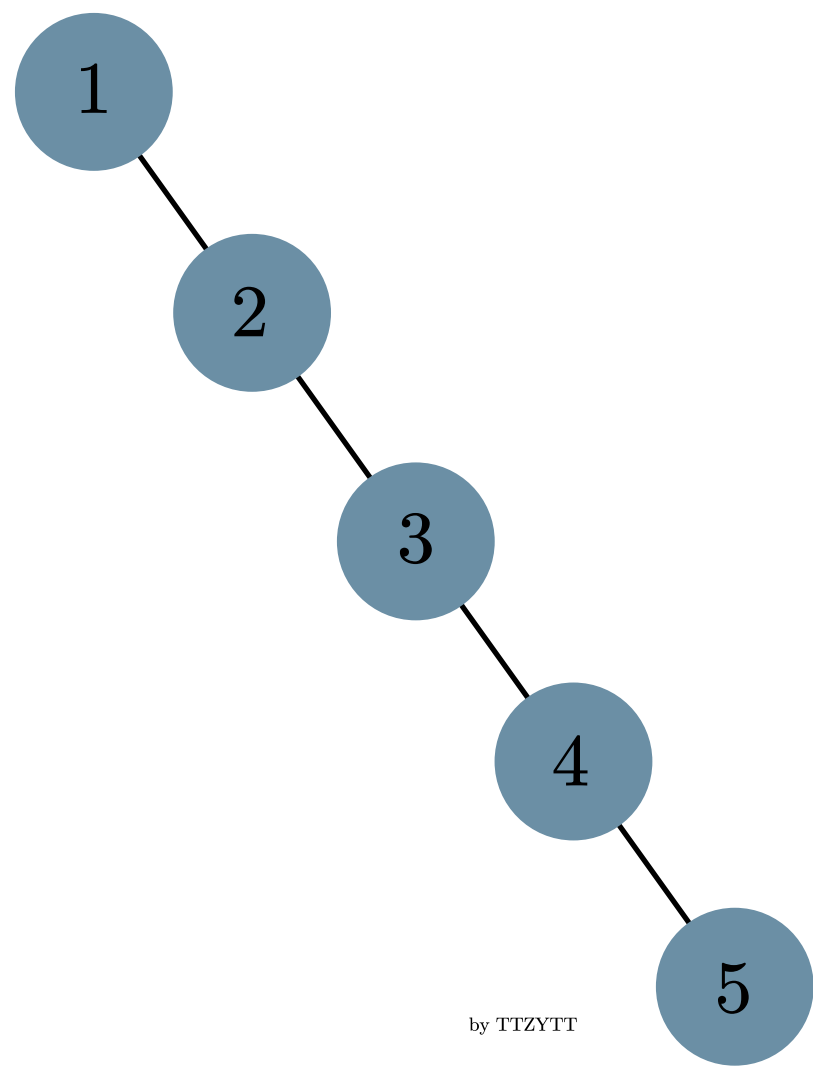
如果插入结点的权值是随机的 (换言之, 是随机插入的), 那么这个朴素搜索树的高度较小 (接近 $\sqrt{n}$, 其中 n 为结点数), 而每一层的节点数较多, 即它的形状会非常的「胖」。

上图的 Treap 就是一个例子。因此此时的任意操作复杂度都将会是 $\sqrt{n}$ 左右。

不过，这只是在随机情况下的复杂度，如果我们按照下面这个非常有序的顺序给一个朴素的搜索树插入节点：

1 1 2 3 4 5

那么这棵树将会退化成链，即变得非常「瘦长」（每次插入的节点都比前面的大，所以都被安排到右子节点了）：



不难看出，查询的复杂度也从 $\sqrt{n}$ 变成了 n 。

而 treap 为了解决这个问题、达到一个较为「平衡」的状态，通过维护随机的优先级满足堆性质，「打乱」了节点的插入顺序，从而让二叉搜索树达到了理想的复杂度，避免了退化成链的问题。

Treap 复杂度的证明

由于 treap 各种操作的复杂度都和所操作的结点的深度有关，我们首先证明，所有结点的期望高度都是 $\sqrt{n}$ 。

记号约定

为了方便表述，我们约定：

- n 是节点个数。
- Treap 结点中满足二叉搜索树性质的称为 **权值**，满足堆性质的（也就是随机的）称为 **优先级**。不妨设优先级满足小根堆性质。
- k 表示权值第 k 小的结点。
- S_k 表示集合 S_k ，即按权值升序排列后第 k 个到第 n 个的结点构成的集合。
- d_k 表示结点 k 的深度。规定根节点的深度是 1。
- X_k 是一个指示器随机变量，当 k 是 k 的祖先时值为 1，否则为 0。特别地， $X_1 = 1$ 。
- P_k 表示事件 $X_k = 1$ 发生的概率。

树高的证明

由于结点 k 的深度等于它祖先的个数，因此有

那么根据期望的线性性，有

由于 是指示器随机变量，它的期望就等于它为 的概率，因此

我们先证明引理：当且仅当 的优先级是 中最小的。

► 引理的证明

那么根据引理，深度的期望可以转化成

又因为结点的优先级是随机的，我们假定集合 中任何一个结点的优先级最小的概率都相同，那么

因此每个结点的期望高度都是 。

而朴素的二叉搜索树的操作的复杂度均是 $O(n)$ ，同时 treap 维护堆性质的复杂度也是 $O(n)$ 的，因此 treap 各种操作的期望复杂度都是 $O(n)$ 。

▼ 期望复杂度的感性理解

首先，我们需要认识到一个节点的 属性是和它所在的层数有直接关联的。再回忆堆的性质：

- 子节点值 (v) 比父节点大或小（取决于是小根堆还是大根堆）

我们发现层数低的节点，比如整个树的根节点，它的 属性也会更小（在小根堆中）。并且，在朴素的搜索树中，先被插入的节点，也更有可能会有比较小的层数。我们可以把这个 属性和被插入的顺序关联起来理解，这样，也就理解了为什么 treap 可以把节点插入的顺序通过 打乱。

给 treap 插入新节点时，需要同时维护树和堆的性质。其中，搜索树的性质可以在插入时维护，而堆性质的维护则有两种处理方法，分别是旋转和分裂、合并。使用这两种方法的 treap 被分别称为 **旋转 treap** 和 **无旋 treap**。

旋转 treap

旋转 treap 维护平衡的方式为旋转，和 AVL 树的旋转操作类似，分为 **左旋** 和 **右旋**。即在满足二叉搜索树的条件下根据堆的优先级对 treap 进行平衡操作。

旋转 treap 在做普通平衡树题的时候，是所有平衡树中常数较小的。

下面的讲解中的代码用指针实现了旋转 treap，文末附有数组形式的完整实现。

▼ Info

代码中的 rank 代表前面讲的优先级（属性），该属性满足的是小根堆性质。

节点结构

```
1 struct Node {
2     Node *ch[2]; // 两个子节点的地址
3     int val, rank;
4     int rep_cnt; // 当前这个值 (val) 重复出现的次数
5     int siz;     // 以当前节点为根的子树大小
6
7     Node(int val) : val(val), rep_cnt(1), siz(1) {
8         ch[0] = ch[1] = nullptr;
9         rank = rand();
10        // 注意初始化的时候，rank 是随机给出的
11    }
12
13    void upd_siz() {
14        // 用于旋转和删除过后，重新计算 siz 的值
15        siz = rep_cnt;
16        if (ch[0] != nullptr) siz += ch[0]->siz;
17        if (ch[1] != nullptr) siz += ch[1]->siz;
18    }
19};
```

旋转

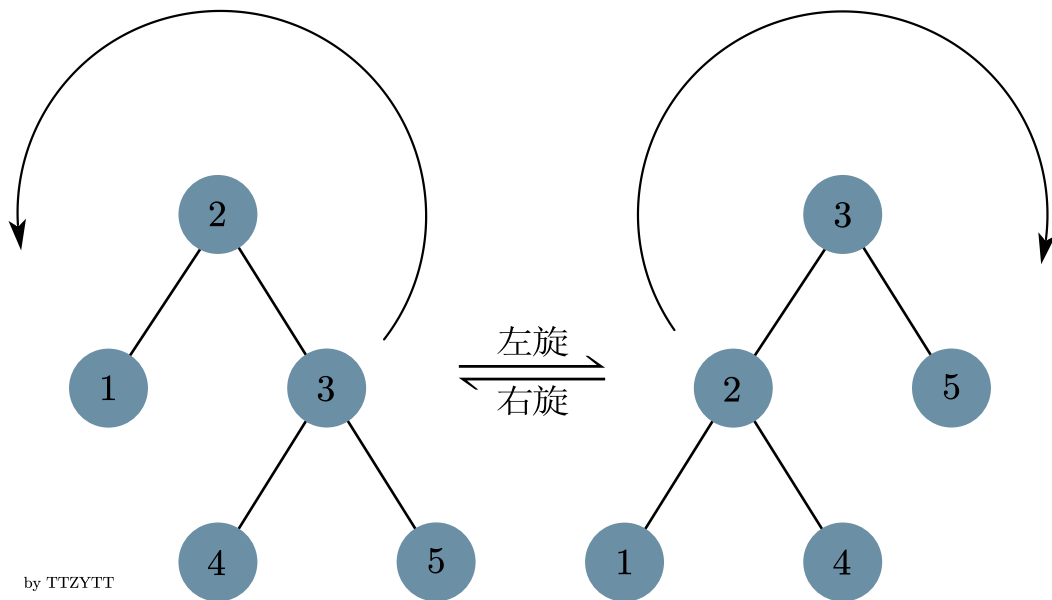
旋转操作是 treap 的一个非常重要的操作，主要用来在保持 treap 树性质的同时，调整不同节点的层数，以达到维护堆性质的作用。

旋转操作的左旋和右旋可能不是特别容易区分，以下是两个较为明显的特点：

旋转操作的含义：

- 在不影响搜索树性质的前提下，把和旋转方向相反的子树变成根节点（如左旋，就是把右子树变成根节点）
- 不影响性质，并且在旋转过后，跟旋转方向相同的子节点变成了原来的根节点（如左旋，旋转完之后的左子节点是旋转前的根节点）

左旋和右旋操作是相互的，如下图。



```

1 enum rot_type { LF = 1, RT = 0 };
2
3 void _rotate(Node *&cur,
4             rot_type dir) { // dir参数代表旋转的方向 0为右旋, 1为左旋
5     // 注意传进来的 cur 是指针的引用, 也就是改了 this
6     // cur, 变量是跟着一起改的, 如果这个 cur 是别的 树的子节点, 根据 ch
7     // 找过来的时候, 也是会找到这里的
8
9     // 以下的代码解释的均是左旋时的情况
10    Node *tmp = cur->ch[dir]; // 让 C 变成根节点,
11                               // 这里的 tmp
12                               // 是一个临时的节点指针, 指向成为新的根节点的节点
13
14    /* 左旋: 也就是让右子节点变成根节点
15     *      A              C
16     *     / \            / \
17     *    B  C  ---->  A  E
18     *           / \    / \
19     *          D  E    B  D
20     */
21    cur->ch[dir] = tmp->ch[!dir]; // 让 A 的右子节点变成 D
22    tmp->ch[!dir] = cur;         // 让 C 的左子节点变成 A
23    cur->upd_siz(), tmp->upd_siz(); // 更新大小信息
24    cur = tmp; // 最后把临时储存 C 树的变量赋值到当前根节点上 (注意 cur 是引用)
25 }

```

插入

类似普通二叉搜索树的插入, 但是需要在插入的过程中通过旋转来维护优先级的堆性质。

```

1 void _insert(Node *&cur, int val) {
2     if (cur == nullptr) {
3         // 没这个节点直接新建
4         cur = new Node(val);
5         return;
6     } else if (val == cur->val) {
7         // 如果有这个值相同的节点, 就把重复数量加一
8         cur->rep_cnt++;
9         cur->siz++;
10    } else if (val < cur->val) {
11        // 维护搜索树性质, val 比当前节点小就插到左边, 反之亦然
12        _insert(cur->ch[0], val);
13        if (cur->ch[0]->rank < cur->rank) {
14            // 小根堆中, 上面节点的优先级一定更小
15            // 因为新插的左子节点比父节点小, 现在需要让左子节点变成父节点
16            _rotate(cur, RT); // 注意前面的旋转性质, 要把左子节点转上来, 需要右旋
17        }
18        cur->upd_siz(); // 插入之后大小会变化, 需要更新
19    } else {
20        _insert(cur->ch[1], val);
21        if (cur->ch[1]->rank < cur->rank) {
22            _rotate(cur, LF);
23        }
24        cur->upd_siz();
25    }
26 }

```

删除

主要就是分类讨论, 不同的情况有不同的处理方法, 删完了树的大小会有变化, 要注意更新。并且如果要删的节点有左子树和右子树, 就要考虑删除之后让谁来当父节点 (维

护 rank 小的节点在上面)。

```
1 void _del(Node *&cur, int val) {
2   if (val > cur->val) {
3     _del(cur->ch[1], val);
4     // 值更大就在右子树, 反之亦然
5     cur->upd_siz();
6   } else if (val < cur->val) {
7     _del(cur->ch[0], val);
8     cur->upd_siz();
9   } else {
10    if (cur->rep_cnt > 1) {
11      // 如果要删除的节点是重复的, 可以直接把重复值减小
12      cur->rep_cnt--, cur->siz--;
13      return;
14    }
15    uint8_t state = 0;
16    state |= (cur->ch[0] != nullptr);
17    state |= ((cur->ch[1] != nullptr) << 1);
18    // 00都无, 01有左无右, 10, 无左有右, 11都有
19    Node *tmp = cur;
20    switch (state) {
21      case 0:
22        delete cur;
23        cur = nullptr;
24        // 没有任何子节点, 就直接把这个节点删了
25        break;
26      case 1: // 有左无右
27        cur = tmp->ch[0];
28        // 把根变成左儿子, 然后把原来的根节点删了, 注意这里的 tmp 是从 cur
29        // 复制的, 而 cur 是引用
30        delete tmp;
31        break;
32      case 2: // 有右无左
33        cur = tmp->ch[1];
34        delete tmp;
35        break;
36      case 3:
37        rot_type dir = cur->ch[0]->rank < cur->ch[1]->rank
38                      ? RT
39                      : LF; // dir 是 rank 更小的那个儿子
40        _rotate(cur, dir); // 这里的旋转可以把优先级更小的儿子转上去, rt 是 0,
41                          // 而 lf 是 1, 刚好跟实际的子树下标反过来
42        _del(
43          cur->ch[!dir],
44          val); // 旋转完成后原来的根节点就在旋方向那边, 所以需要
45              // 继续把这个原来的根节点删掉
46              // 如果说要删的这个节点是在整个树的「上层的」, 那我们会一直通过这
47              // 这里的旋转操作, 把它转到没有子树了 (或者只有一个), 再删掉它。
48        cur->upd_siz();
49        // 删除会造成大小改变
50        break;
51    }
52  }
53 }
```

根据值查询排名

操作含义: 查询以 cur 为根节点的子树中, val 这个值的大小的排名 (该子树中小于 val 的节点的个数 + 1)

```
1 int _query_rank(Node *cur, int val) {
2   int less_siz = cur->ch[0] == nullptr ? 0 : cur->ch[0]->siz;
3   // 这个树中小于 val 的节点的数量
4   if (val == cur->val)
5     // 如果这个节点就是要查的节点
6     return less_siz + 1;
7   else if (val < cur->val) {
8     if (cur->ch[0] != nullptr)
9       return _query_rank(cur->ch[0], val);
10    else
11      return 1; // 如果左子树是空的, 说比最小的节点还要小, 那这个数字就是最小的
12  } else {
13    if (cur->ch[1] != nullptr)
14      // 如果要查的值比这个节点大, 那这个节点的左子树以及这个节点自身肯定都比要查的值小
15      // 所以要加上这两个值, 再加上往右边找的结果
16      // (以右子树为根的子树中, val 这个值的大小的排名)
17      return less_siz + cur->rep_cnt + _query_rank(cur->ch[1], val);
18    else
19      return cur->siz + 1;
20    // 没有右子树的话直接整个树 + 1 相当于 less_siz + cur->rep_cnt + 1
21  }
22 }
```

根据排名查询值

要根据排名查询值, 我们首先要知道如何判断要查的节点在树的哪个部分:

以下是一个判断方法的表:

左子树	根节点/当前节点	右子树
排名 $\leq$ 左子树的大小	排名 $>$ 左子树的大小，并且 $\leq$ 左子树的大小 + 根节点的重复次数	排名 $>$ 左子树的大小 + 根节点的重复次数

注意如果在右子树，递归的时候需要对原来的 `rank` 进行处理。递归的时候就相当去查，在右子树中为这个排名的值，为了把排名转换成基于右子树的，需要把原来的 `rank` 减去左子树的大小和根节点的重复次数。

可以把所有节点想象成一个排好序的数组，或者数轴（如下），

```

1 1 -> |左子树的节点|根节点|右子树的节点| -> n
2           ^
3           要查的排名
4           ↓转换成基于右子树的排名
5 1 -> |右子树的节点| -> n
6           ^
7           要查的排名

```

这里的转换方法就是直接把排名减去左子树的大小和根节点的重复数量。

```

1 int _query_val(Node *cur, int rank) {
2     // 查询树中第 rank 大的节点的值
3     int less_siz = cur->ch[0] == nullptr ? 0 : cur->ch[0]->siz;
4     // less_siz 是左子树的大小
5     if (rank <= less_siz)
6         return _query_val(cur->ch[0], rank);
7     else if (rank <= less_siz + cur->rep_cnt)
8         return cur->val;
9     else
10        return _query_val(cur->ch[1], rank - less_siz - cur->rep_cnt); // 见前文
11 }

```

查询第一个比 val 小的节点

注意这里使用了一个类中的全局变量，`q_prev_tmp`。

这个值是在 `val` 比当前节点值大的时候才会被更改的，所以返回这个变量就是返回 `val` 最后一次比当前节点的值大，之后就是更小了。

```

1 int _query_prev(Node *cur, int val) {
2     if (val <= cur->val) {
3         // 还是比 val 大，所以往左子树找
4         if (cur->ch[0] != nullptr) return _query_prev(cur->ch[0], val);
5     } else {
6         // 只有能进到这个 else 里，才会更新 q_prev_tmp 的值
7         q_prev_tmp = cur->val;
8         // 当前节点已经比 val 小了，但是不确定是否是最大的，所以要向右子树继续找
9         if (cur->ch[1] != nullptr) _query_prev(cur->ch[1], val);
10        // 接下来的递归可能不会更改 q_prev_tmp
11        // 了，那就直接返回这个值，总之返回的就是最后一次进到 这个 else 中的
12        // cur->val
13        return q_prev_tmp;
14    }
15    return NIL;
16 }

```

查询第一个比 val 大的节点

跟前一个很相似，只是大于小于号换了一下。

```

1 int _query_nex(Node *cur, int val) {
2     if (val >= cur->val) {
3         if (cur->ch[1] != nullptr) return _query_nex(cur->ch[1], val);
4     } else {
5         q_nex_tmp = cur->val;
6         if (cur->ch[0] != nullptr) _query_nex(cur->ch[0], val);
7         return q_nex_tmp;
8     }
9     return NIL;
10 }

```

无旋 treap

无旋 `treap` 的操作方式使得它天生支持维护序列、可持久化等特性。

无旋 `treap` 又称分裂合并 `treap`。它仅有两种核心操作，即为 **分裂** 与 **合并**。通过这两种操作，在很多情况下可以比旋转 `treap` 更方便的实现别的操作。下面逐一介绍这两种操作。

▼ 注释

讲解无旋 `treap` 应当提到 **FHQ-Treap**（by 范浩强）。即可持久化，支持区间操作的无旋 `Treap`。更多内容请参照《范浩强谈数据结构》ppt。

分裂（split）

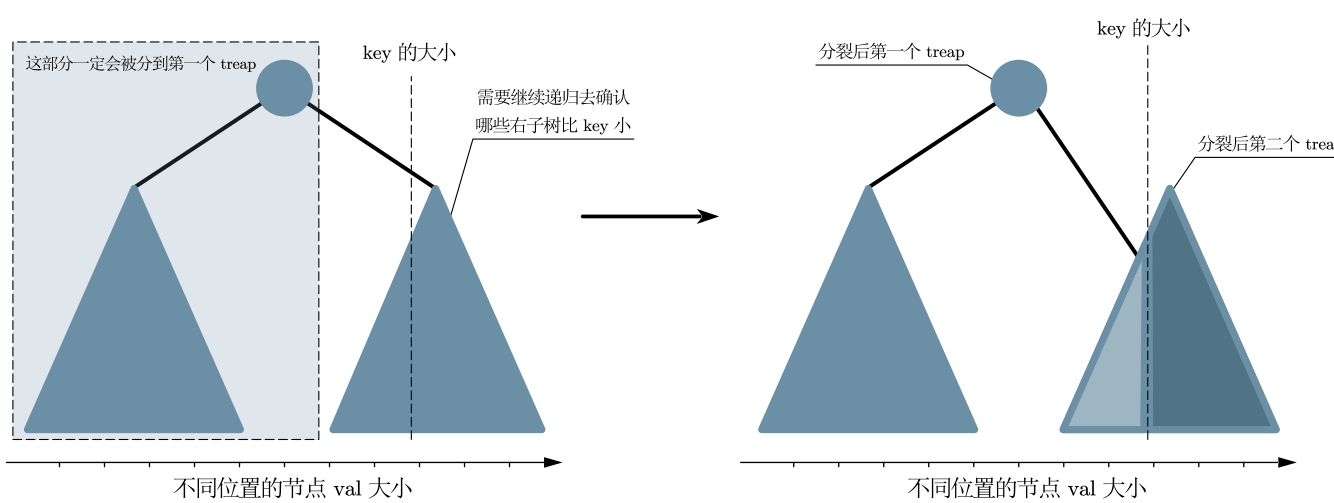
按值分裂

分裂过程接受两个参数：根指针、关键值。结果为将根指针指向的 treap 分裂为两个 treap，第一个 treap 所有结点的值 0 小于等于，第二个 treap 所有结点的值大于。

该过程首先判断 是否小于 的值，若小于，则说明 及其右子树全部大于，属于第二个 treap。当然，也可能有一部分的左子树的值大于，所以还需要继续向左子树递归地分裂。对于大于 的那部分左子树，我们把它作为 的左子树，这样，整个 上的节点都是大于 的。

相应的，如果 大于等于 的值，说明 的整个左子树以及其自身都小于，属于分裂后的第一个 treap。并且， 的部分右子树也可能有部分小于，因此我们需要继续递归地分裂右子树。把小于 的那部分作为 的右子树，这样，整个 上的节点都小于。

下图展示了 的值小于等于 时按值分裂的情况。¹



by TTYZTT

if $cur \rightarrow val \leq key$

```
1 pair<Node *, Node *> split(Node *cur, int key) {
2   if (cur == nullptr) return {nullptr, nullptr};
3   if (cur->val <= key) {
4     // cur 以及它的左子树一定属于分裂后的第一个树
5     auto temp = split(cur->ch[1], key);
6     // 但是它可能有部分右子树也比 key 小
7     cur->ch[1] = temp.first;
8     // 我们把小于 key 的那部分拿出来，作为 cur 的右子树，这样整个 cur 都是小于
9     // key 的 剩下的那部分右子树成为分裂后的第二个 treap
10    cur->upd_siz();
11    // 分裂过后树的大小会变化，需要更新
12    return {cur, temp.second};
13  } else {
14    // 同上
15    auto temp = split(cur->ch[0], key);
16    cur->ch[0] = temp.second;
17    cur->upd_siz();
18    return {temp.first, cur};
19  }
20 }
```

按排名分裂

比起按值分裂，这个操作更像是旋转 treap 中的根据排名（某个节点的排名是树中所有小于此节点值的节点的数量）查询值：

此函数接受两个参数，节点指针 和排名，返回分裂后的三个 treap。

其中，第一个 treap 中每个节点的排名都小于，第二个的排名等于，并且第二个 treap 只有一个节点（不可能有多个等于的，如果有的话会增加 Node 结构体中的 cnt），第三个则是大于。

此操作的重点在于判断排名和 相等的节点在树的哪个部分，这也是旋转 treap 根据排名查询值操作时的重要部分，在前文有非常详细的解释，这里不过多讲解。

并且，此操作的递归部分和按值分裂也非常相似，这里不赘述。

```

1 tuple<Node *, Node *, Node *> split_by_rk(Node *cur, int rk) {
2   if (cur == nullptr) return {nullptr, nullptr, nullptr};
3   int ls_siz = cur->ch[0] == nullptr ? 0 : cur->ch[0]->siz;
4   if (rk <= ls_siz) {
5     // 排名和 cur 相等的节点在左子树
6     Node *l, *mid, *r;
7     tie(l, mid, r) = split_by_rk(cur->ch[0], rk);
8     cur->ch[0] = r; // 返回的第三个 treap 中的排名都大于 rk
9     // cur 的左子树被设成 r 后, 整个 cur 中节点的排名都大于 rk
10    cur->upd_siz();
11    return {l, mid, cur};
12  } else if (rk <= ls_siz + cur->cnt) {
13    // 和 cur 相等的就是当前节点
14    Node *lt = cur->ch[0];
15    Node *rt = cur->ch[1];
16    cur->ch[0] = cur->ch[1] = nullptr;
17    // 分裂后第二个 treap 只有一个节点, 所有要把它的子树设置为空
18    return {lt, cur, rt};
19  } else {
20    // 排名和 cur 相等的节点在右子树
21    // 递归过程同上
22    Node *l, *mid, *r;
23    tie(l, mid, r) = split_by_rk(cur->ch[1], rk - ls_siz - cur->cnt);
24    cur->ch[1] = l;
25    cur->upd_siz();
26    return {cur, mid, r};
27  }
28 }

```

合并 (merge)

合并过程接受两个参数：左 treap 的根指针、右 treap 的根指针。必须满足 中所有结点的值小于等于 中所有结点的值。一般来说，我们合并的两个 treap 都是原来从一个 treap 中分裂出去的，所以不难满足 中所有节点的值都小于

在旋转 treap 中，我们借助旋转操作来维护 符合堆的性质，同时旋转时还不能改变树的性质。在无旋 treap 中，我们用合并达到相同的效果。

因为两个 treap 已经有序，所以我们在合并的时候只需要考虑把哪个树「放在上面」，把哪个「放在下面」，也就是是需要判断将哪个树作为子树。显然，根据堆的性质，我们需要把 小的放在上面（这里采用小根堆）。

同时，我们还需要满足搜索树的性质，所以若 的根结点的 小于 的，那么 即为新根结点，并且 因为值比 更大，应与 的右子树合并；反之，则 作为新根结点，然后因为 的值比 小，与 的左子树合并。

```

1 Node *merge(Node *u, Node *v) {
2   // 传进来的两个树的内部已经符合搜索树的性质了
3   // 并且 u 内所有节点的值 < v 内所有节点的值
4   // 所以在合并的时候需要维护堆的性质
5   // 这里用的是小根堆
6   if (u == nullptr && v == nullptr) return nullptr;
7   if (u != nullptr && v == nullptr) return u;
8   if (v != nullptr && u == nullptr) return v;
9
10  if (u->prio < v->prio) {
11    // u 的 prio 比较小, u应该作为父节点
12    u->ch[1] = merge(u->ch[1], v);
13    // 因为 v 比 u 大, 所以把 v 作为 u 的右子树
14    u->upd_siz();
15    return u;
16  } else {
17    // v 比较小, v应该作为父节点
18    v->ch[0] = merge(u, v->ch[0]);
19    // u 比 v 小, 所以递归时的参数是这样的
20    v->upd_siz();
21    return v;
22  }
23 }

```

插入

在无旋 treap 中，插入，删除，根据值查询排名等基础操作既可以用普通二叉查找树的方法实现，也可以用分裂和合并来实现。通常来说，使用分裂和合并来实现更加简洁，但是速度会慢一点^[2]。为了帮助更好的理解无旋 treap，下面的操作全部使用分裂和合并实现。

在实现插入操作时，我们利用了分裂操作的一些性质。也就是值小于等于 的节点会被分到第一个 treap。

所以，假设我们根据 分裂当前这个 treap。会有下面两棵树，并符合以下条件：

其中 表示分裂后所有被分到第一个 treap 的节点的集合，则是第二个。

如果我们再按照 继续分裂，那么会产生下面两棵树，并符合以下条件：

其中 表示 分裂后所有被分到第一个 treap 的节点的集合，则是第二个。并且上面的式子中，后半部分的 来自于 所符合的条件。

不难发现，只要 和节点的值是一个整数（大多数使用场景下会使用整数）那么符合 条件的节点只有一个，也就是值等于 的节点。

在插入时，如果我们发现符合 的节点存在，那就可以直接增加重复次数，否则，就新开一个节点。

注意把树分裂好了还需要用合并操作把它「粘」回去，这样下次还能继续使用。并且，还需要注意合并操作的参数顺序是有要求的，第一个树的所有节点的值都需要小于第二个。

```
1 void insert(int val) {
2     auto temp = split(root, val);
3     // 根据 val 的值把整个树分成两个
4     // 注意 split 的实现，等于 val 的子树是在左子树的
5     auto l_tr = split(temp.first, val - 1);
6     // l_tr 的左子树 <= val - 1, 如果有 = val 的节点，那一定在右子树
7     Node *new_node;
8     if (l_tr.second == nullptr) {
9         // 没有这个节点就新开，否则直接增加重复次数。
10        new_node = new Node(val);
11    } else {
12        l_tr.second->cnt++;
13        l_tr.second->upd_siz();
14    }
15    Node *l_tr_combined =
16        merge(l_tr.first, l_tr.second == nullptr ? new_node : l_tr.second);
17    // 合并 T_1 left 和 T_1 right
18    root = merge(l_tr_combined, temp.second);
19    // 合并 T_1 和 T_2
20 }
```

删除

删除操作也使用和插入操作相似的方法，找到值和 相等的节点，并且删除它。

```
1 void del(int val) {
2     auto temp = split(root, val);
3     auto l_tr = split(temp.first, val - 1);
4     if (l_tr.second->cnt > 1) {
5         // 如果这个节点的重复次数大于 1, 减小即可
6         l_tr.second->cnt--;
7         l_tr.second->upd_siz();
8         l_tr.first = merge(l_tr.first, l_tr.second);
9     } else {
10        if (temp.first == l_tr.second) {
11            // 有可能整个 T_1 只有这个节点，所以也需要把这个点设成 null 来标注已经删除
12            temp.first = nullptr;
13        }
14        delete l_tr.second;
15        l_tr.second = nullptr;
16    }
17    root = merge(l_tr.first, temp.second);
18 }
```

根据值查询排名

排名是比这个值小的节点的数量，所以我们根据 分裂当前树，那么分裂后的第一个树就符合：

如果树的值和 为整数，那么 就包含了所有值小于 的节点。

```
1 int qrank_by_val(Node* cur, int val) {
2     auto temp = split(cur, val - 1);
3     int ret = (temp.first == nullptr ? 0 : temp.first->siz) + 1; // 根据定义 + 1
4     root = merge(temp.first, temp.second); // 拆好了再粘回去
5     return ret;
6 }
```

根据排名查询值

调用 split_by_rk() 函数后，会返回分裂好的三个 treap，其中第二个只包含一个节点，它的排名等于，所以我们直接返回这个节点的。

```
1 int qval_by_rank(Node *cur, int rk) {
2     Node *l, *mid, *r;
3     tie(l, mid, r) = split_by_rk(cur, rk);
4     int ret = mid->val;
5     root = merge(merge(l, mid), r);
6     return ret;
7 }
```

查询第一个比 val 小的节点

可以把这个问题转化为，在比 小的所有节点中，找出排名最大的。我们根据 来分裂这个 treap，返回的第一个 treap 中的节点的值就全部小于，然后我们调用 qval_by_rank() 找出这个树中值最大的节点。

```
1 int qprev(int val) {
2     auto temp = split(root, val - 1);
3     // temp.first 就是值小于 val 的子树
4     int ret = qval_by_rank(temp.first, temp.first->siz);
5     // 这里查询的是，所有小于 val 的节点里面，最大的那个值
6     root = merge(temp.first, temp.second);
7     return ret;
8 }
```

查询第一个比 val 大的节点

和上个操作类似，可以把这个问题转化为，在比 val 大的所有节点中，找出排名最小的。那么根据 split 后，返回的第二个 treap 中的所有节点的值就大于 val。

然后我们去查询这个树中排名为 k 的节点（也就是值最小的节点）的值，就可以成功查到第一个比 val 大的节点。

```
1 int qnex(int val) {
2     auto temp = split(root, val);
3     int ret = qval_by_rank(temp.second, 1);
4     // 查询所有大于 val 的子树里面，值最小的那个
5     root = merge(temp.first, temp.second);
6     return ret;
7 }
```

建树 (build)

将一个有 n 个节点的序列 a 转化为一棵 treap。

可以依次暴力插入这 n 个节点，每次插入一个权值为 a[i] 的节点时，将整棵 treap 按照权值分裂成权值小于等于 a[i] 的和权值大于 a[i] 的两部分，然后新建一个权值为 a[i] 的节点，将两部分和新节点按从小到大的顺序依次合并，单次插入时间复杂度为 $O(n)$ ，总时间复杂度为 $O(n^2)$ 。

在某些题目内，可能会有多次插入一段有序序列的操作，这是就需要在 $O(n \log n)$ 的时间复杂度内完成建树操作。

方法一：在递归建树的过程中，每次选取当前区间的中点作为该区间的树根，并对每个节点钦定合适的优先值，使得新树满足堆的性质。这样能保证树高为 $O(\log n)$ 。

方法二：在递归建树的过程中，每次选取当前区间的中点作为该区间的树根，然后给每个节点一个随机优先级。这样能保证树高为 $O(\log n)$ ，但不保证其满足堆的性质。这样也是正确的，因为无旋式 treap 的优先级是用来使 merge 操作更加随机一点，而不是用来保证树高的。

方法三：观察到 treap 是笛卡尔树，利用笛卡尔树的建树方法即可，用单调栈维护右链即可。

无旋 treap 的区间操作

建树

无旋 treap 相比旋转 treap 的一大好处就是可以实现各种区间操作，下面我们以文艺平衡树的 [模板题](#) 为例，介绍 treap 的区间操作。

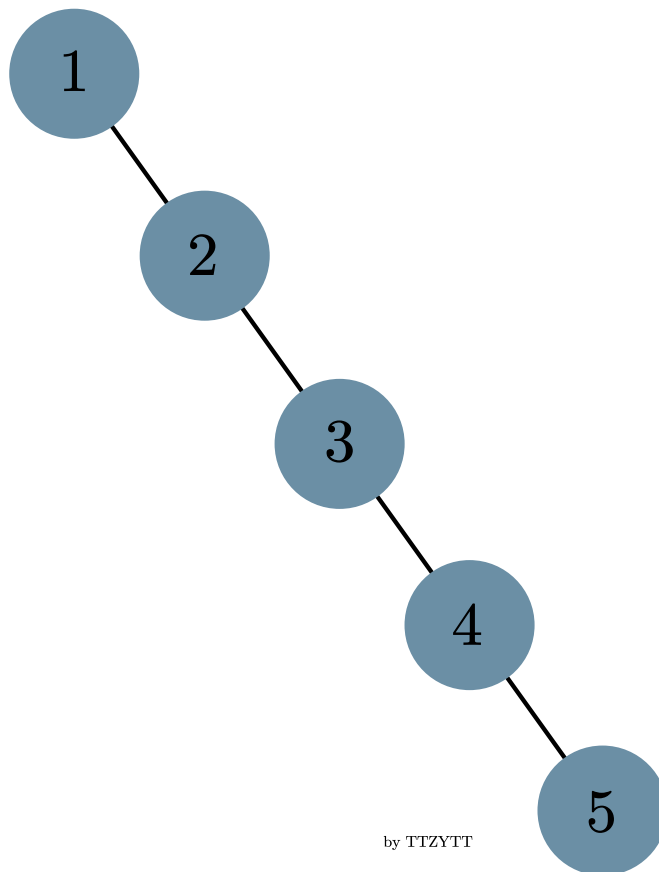
您需要写一种数据结构（可参考题目标题），来维护一个有序数列。

其中需要提供以下操作：翻转一个区间，例如原有序序列是 1 2 3 4 5，翻转区间是 [2, 4] 的话，结果是 1 4 3 2 5。对于 n 的数据，（初始区间长度）（翻转次数）

在这道题目中，我们需要实现的是区间翻转，那么我们首先需要考虑如何建树，建出来的树需要是初始的区间。

我们只需要把区间的下标依次插入 treap 中，这样在中序遍历（先遍历左子树，然后当前节点，最后右子树）时，就可以得到这个区间³。

我们知道在朴素的二叉查找树中按照递增的顺序插入节点，建出来的树是一个长链，按照中序遍历，自然可以得到这个区间。



如上图，按照 的顺序给朴素搜索树插入节点，中序遍历时，得到的也是 。

但是在 treap 中，按增序插入节点后，在合并操作时还会根据 调整树的结构，在这样的情况下，如何确保中序遍历一定能正确的输出呢？

可以参考 [笛卡尔树的单调栈建树方法](#) 来理解这个问题。

设新插入的节点为 。

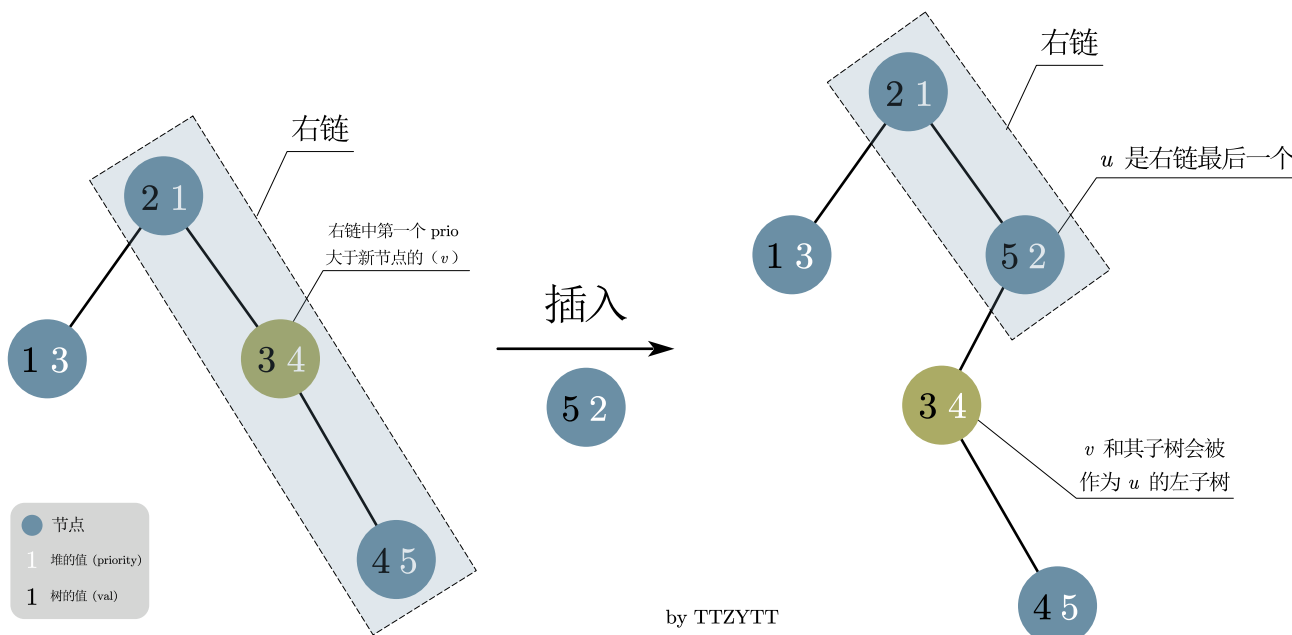
首先，因为是递增地插入节点，每一个新插入的节点肯定会被连接到 treap 的右链（即从根结点一直往右子树走，经过的结点形成的链）上。

从根节点开始，右链上的节点的 是递增的（小根堆）。那我们可以找到右链上第一个 大于 的节点，我们叫这个节点，并把这个节点换成 。

因为 一定大于这个树上其他的全部节点，我们需要把 以及它的子树作为 的左子树。并且此时 没有右子树。

可以发现，中序遍历时 一定是最后一个被遍历到的（因为 是右链中的最后一个，而中序遍历中，右子树是最后被遍历到的）。

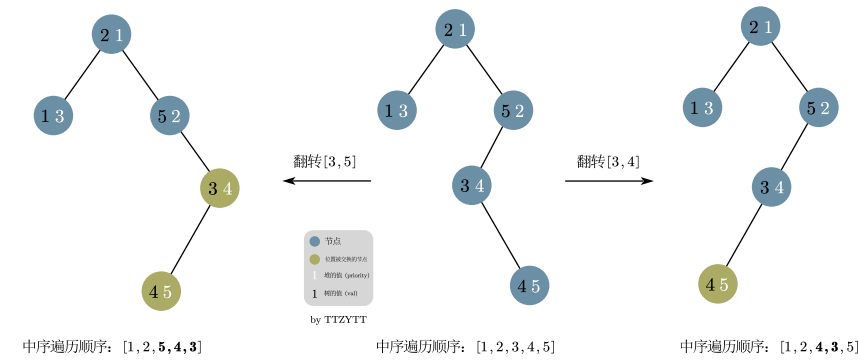
下图是一个 treap 根据递增顺序插入 号节点时，插入 号节点时的变化，可以用这张图更好的理解按照增序插入的过程。



区间翻转

翻转 这个区间时，基本思路是将树分裂成 三个区间，再对中间的 进行翻转³。

翻转的具体操作是把区间内的子树的每一个左，右子节点交换位置。如下图就展示了翻转上图中 `treap` 的 和 区间后的 `treap`。



注意如果按照这个方法翻转，那么每次翻转 区间时，就会有 个节点会被交换位置，这样频繁的操作显然不能满足 的数据范围，其 的单次翻转复杂度甚至不如暴力（因为我们除了需要花线性时间交换节点外，还需要在树中花费 的时间找到需要交换的节点）。

再观察题目要求，可以发现因为只需要最后输出操作完的区间，所以并不需要每次都真的去交换。如此一来，便可以使用线段树中常用的懒标记（lazy tag）来优化复杂度。交换时，只需要在父节点打上标记，代表这个子树下的每个左右子节点都需要交换就行了。

在线段树中，我们一般在更新和查询时上传懒标记。这是因为，在更新和查询时，我们想要更新/查询的范围不一定和懒标记代表的范围重合，所以要先上传标记，确保查到和更新后的值是正确的。

在无旋 `treap` 中也是一样。具体操作时我们会把 `treap` 分裂成前文讲到的三个树，然后给中间的树打上懒标记后合并这三棵树。因为我们想要翻转的区间和懒标记代表的区间不一定重合，所以要在分裂时上传标记。并且，分裂和合并操作会造成每个节点及其懒标记所代表的节点发生变动，所以也需要在合并前上传懒标记。

换句话说，是当树的结构发生改变的时候，当我们进行分裂或合并操作时需要改变某一个点的左右儿子信息时之前，应该下放标记，而非之后，因为懒标记是需要下传给儿子节点的，但更改左右儿子信息之后若懒标记还未下放，则懒标记就丢失了下放的对象。⁴

以下为代码讲解，代码参考了³。

因为区间操作中大部分操作都和普通的无旋 `treap` 相同，所以这里只讲解和普通无旋 `treap` 不同的地方。

下传标记

需要注意这里的懒标记代表需要把这个树中的每一个子节点交换位置。所以如果当前节点的子节点也有懒标记，那两次翻转就抵消了。如果子节点不需要翻转，那么这个懒标记就需要继续被下传到子节点上。

```
1 // 这里这个 pushdown 是 Node 类的成员函数，其中 to_rev 是懒标记
2 void pushdown() {
3     swap(ch[0], ch[1]);
4     if (ch[0] != nullptr) ch[0]->to_rev ^= 1;
5     if (ch[1] != nullptr) ch[1]->to_rev ^= 1;
6     to_rev = false;
7 }
8
9 void check_tag() {
10     if (to_rev) pushdown();
11 }
```

分裂

注意在这个题目中，因为翻转操作，`treap` 中的 会不符合二叉搜索树的性质（见区间翻转部分的图），所以我们不能根据 来判断应该往左子树还是右子树递归。

所以这里的分裂跟普通无旋 `treap` 中的按排名分裂更相似，是根据当前树的大小判断往左还是右子树递归的，换言之，我们是按照开始时这个节点在树中的位置来判断的。

返回的第一个 `treap` 中节点的排名全部小于等于，而第二个 `treap` 中节点的排名则全部大于。

```

1 #define siz(_) (_ == nullptr ? 0 : _->siz)
2
3 pair<Node*, Node*> split(Node* cur, int sz) {
4     // 按照树的大小判断
5     if (cur == nullptr) return {nullptr, nullptr};
6     cur->check_tag();
7     // 分裂前先下传
8     if (sz <= siz(cur->ch[0])) {
9         auto temp = split(cur->ch[0], sz);
10        cur->ch[0] = temp.second;
11        cur->upd_siz();
12        return {temp.first, cur};
13    } else {
14        auto temp =
15            split(cur->ch[1],
16                sz - siz(cur->ch[0]) -
17                1); // 这里的转换在有旋 treap 的 「根据排名查询值有讲」
18        cur->ch[1] = temp.first;
19        cur->upd_siz();
20        return {cur, temp.second};
21    }
22 }

```

合并

唯一需要注意的是在合并前下传懒标记

```

1 Node *merge(Node *sm, Node *bg) {
2     // small, big
3     if (sm == nullptr && bg == nullptr) return nullptr;
4     if (sm != nullptr && bg == nullptr) return sm;
5     if (sm == nullptr && bg != nullptr) return bg;
6     sm->check_tag(), bg->check_tag();
7     if (sm->prio < bg->prio) {
8         sm->ch[1] = merge(sm->ch[1], bg);
9         sm->upd_siz();
10        return sm;
11    } else {
12        bg->ch[0] = merge(sm, bg->ch[0]);
13        bg->upd_siz();
14        return bg;
15    }
16 }

```

区间翻转

和前面介绍的一样，分裂出 三个区间，然后对中间的区间打上标记后再合并。

```

1 void seg_rev(int l, int r) {
2     // 这里的 less 和 more 是相对于 l 的
3     auto less = split(root, l - 1);
4     // 所有小于等于 l - 1 的会在 less 的左子树
5     auto more = split(less.second, r - l + 1);
6     // 从 l 开始的前 r - l + 1 个元素的区间
7     more.first->to_rev = true;
8     root = merge(less.first, merge(more.first, more.second));
9 }

```

中序遍历打印

要注意在打印时要下传标记。

```

1 void print(Node* cur) {
2     if (cur == nullptr) return;
3     cur->check_tag();
4     // 中序遍历 -> 先左子树，再自己，最后右子树
5     print(cur->ch[0]);
6     cout << cur->val << " ";
7     print(cur->ch[1]);
8 }

```

完整代码

旋转 treap

指针实现

► 完整代码

数组实现

以下是 bzoj 普通平衡树模板代码，使用数组实现。

► 完整代码

无旋 treap

指针实现

► 完整代码

无旋 treap 的区间操作

指针实现

► 完整代码

例题

[普通平衡树](#)

[文艺平衡树 \(Splay\)](#)

[「ZJOI2006」书架](#)

[「NOI2005」维护数列](#)

[CF 702F T-Shirts](#)

参考资料与注释

-
1. 本图的设计参考了 [维基百科 treap 词条的配图](#) ↩
 2. <https://charleswu.site/archives/1051> ↩
 3. <https://www.cnblogs.com/Equinox-Flower/p/10785292.html> ↩↩↩
 4. <https://www.luogu.com.cn/blog/85514/fhq-treap-xue-xi-bi-ji> ↩
-

本页面最近更新：2025/2/2 12:37:17，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[Dev-XYS](#), [qwqAutomaton](#), [Sora233](#), [ttzytt](#), [CCXXXI](#), [cesonic](#), [ChungZH](#), [Early0v0](#), [Enter-tainer](#), [Henry-ZHR](#), [HeRaNO](#), [HRiver2](#), [hsfzLZH1](#), [lr1d](#), [isdanni](#), [kenlig](#), [lleiix](#), [lychees](#), [Menci](#), [mgt](#), [ouuan](#), [Q-wjh213](#), [Qiyuan Gu](#), [scp020](#), [shuzhouliu](#), [StudyingFather](#), [Tiphereth-A](#), [TrisolarisHD](#), [tsawke](#), [untitledunrevised](#), [zcz0263](#), [zhcphu](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用